JavaScript classes cheat sheet (concise, study-ready)

Use this as a quick reference for class syntax, instantiation, property/method access, inheritance, static members, getters/setters, privacy, and “this” binding.

Core pattern: define a class, instantiate, access

Js Copy

```
// Class definition
class Person {
  // Public field initializers (run per instance)
  firstName = '';
  lastName = '';
  role = 'guest';

  // Constructor
  constructor(firstName, lastName) {
    this.firstName = firstName;
    this.lastName = lastName;
  }

  // Instance method (access with this)
  fullName() {
    return `${this.firstName} ${this.lastName}`;
  }

  // Instance method using parameters and internal state
  introduce(prefix = 'Hi') {
    return `${prefix}, I'm ${this.fullName()} (${this.role}).`;
  }
}

// Instantiate (create objects)
const p1 = new Person('Yasser', 'Hassan');

// Access properties (outside)
p1.role = 'admin';           // dot notation
p1['firstName'] = 'Y.';      // bracket notation (dynamic keys supported)

// Call methods (outside)
console.log(p1.fullName());          // "Y. Hassan"
console.log(p1.introduce('Hello'));  // "Hello, I'm Y. Hassan (admin)."
```

Getters, setters, computed names, method chaining

Js Copy

```
class Account {
  balance = 0;

  constructor(owner, initial = 0) {
    this.owner = owner;
    this.balance = initial;
  }

  // Getter: read as a property
  get isOverdrawn() {
    return this.balance < 0;
  }

  // Setter: write as a property with validation
  set setBalance(value) {
    if (!Number.isFinite(value)) throw new TypeError('Balance must be a number');
    this.balance = value;
  }

  // Chainable methods (return this)
  deposit(amount) {
    this.balance += amount;
    return this;
  }
  withdraw(amount) {
    this.balance -= amount;
    return this;
  }

  // Computed method name
  ['summary']() {
    return `${this.owner}: ${this.balance}`;
  }
}

const a = new Account('Yasser', 100)
  .deposit(50)
  .withdraw(30);

console.log(a.summary());     // "Yasser: 120"
console.log(a.isOverdrawn);   // false
a.setBalance = 200;           // uses setter
```

Static fields and methods (class-level, not per instance)

Js Copy

```
class IdGenerator {
  static prefix = 'ID';
  static #seq = 0; // private static

  static next() {
    return `${this.prefix}-${++this.#seq}`;
  }

  static from(str) {
    return `${this.prefix}-${str}`;
  }
}

console.log(IdGenerator.next());   // "ID-1"
console.log(IdGenerator.from('X')) // "ID-X"

// Note: call static via ClassName.method(), not instance.method().
```

True private fields and methods

Js Copy

```
class SecretBox {
  #secret;               // private field (accessible only inside the class)
  #hits = 0;             // another private field
  static #salt = 's1';   // private static

  constructor(secret) {
    this.#secret = secret;
  }

  #touch() {             // private method
    this.#hits++;
  }

  reveal(mask = false) {
    this.#touch();
    return mask ? '***' : `${this.#secret}:${SecretBox.#salt}:${this.#hits}`;
  }
```

```
}

const box = new SecretBox('token');
console.log(box.reveal());    // "token:s1:1"
console.log(box.reveal(true)) // "***"
// box.#secret; // SyntaxError — cannot access
```

Inheritance, super, overriding

Js Copy

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    return `${this.name} makes a sound.`;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);           // call parent constructor first
    this.breed = breed;
  }
  speak() {
    const base = super.speak(); // call parent method
    return `${base} ${this.name} barks!`;
  }
}

const d = new Dog('Rex', 'Husky');
console.log(d.speak()); // "Rex makes a sound. Rex barks!"
```

“this” binding and arrow methods

Js Copy

```
class ButtonHandler {
  count = 0;
```

```
  // Regular method: loses this if detached
  increment() {
    this.count++;
    return this.count;
  }

  // Arrow property: lexically binds this (good for callbacks)
  incrementBound = () => {
    this.count++;
    return this.count;
  }
}

const h = new ButtonHandler();

// Detaching methods
const f1 = h.increment;
const f2 = h.incrementBound;

try {
  f1(); // this is undefined in strict mode => TypeError
} catch (e) {
  console.log('f1 failed as expected');
}

console.log(f2()); // works, increments to 1
```

Property access patterns and safety

Js Copy

```
const user = { profile: { name: 'Yasser', tags: ['dev'] } };

// Dot vs bracket
console.log(user.profile.name);         // dot (static key)
const key = 'name';
console.log(user.profile[key]);         // bracket (dynamic key)

// Optional chaining (safe navigation)
console.log(user.profile?.address?.city); // undefined (no crash)

// Nullish coalescing (default only for null/undefined)
const nickname = user.profile.nickname ?? 'No nick'; // "No nick"

// Existence checks
console.log('profile' in user);         // true
```

```
console.log(user.hasOwnProperty('x'));  // false

// Destructuring
const { profile: { name, tags = [] } } = user;
console.log(name, tags[0]); // "Yasser", "dev"
```

Patterns you'll use often

- **Class fields** initialize per-instance state without boilerplate in constructor.
- **Return this** for chainable APIs.
- **Getters/setters** create clean read/write interfaces with validation.
- **Static members** hold utilities and factory methods.
- **Private fields (#name)** enforce encapsulation at runtime.
- **Arrow methods in classes** auto-bind this for event handlers/callbacks.
- **super** lets subclasses reuse and extend parent behavior.
- **Optional chaining + nullish coalescing** keep access safe and defaults precise.